

API, Debugging, And Project Explanation

Today we run a small FastAPI backend and explain it like an entry-level technical project.

Pieces we connect

- Linux terminal
- Python
- FastAPI
- SQLite
- JSON
- curl or browser docs

Final project

- GET /health
- GET /tickets
- POST /tickets
- GET /tickets/{ticket_id}
- automatic docs at /docs

How The Pieces Fit

An API is a bridge between software clients and backend data.

Browser, docs page, or curl

- > HTTP request
 - > FastAPI endpoint
 - > Python function
 - > SQLite query
 - > JSON response

Students should be able to explain this flow in plain English.

Website Vs API

Both use the web, but they serve different main purposes.

Website

- human reads pages
- HTML and CSS
- buttons and forms
- visual layout matters

API

- software reads data
- JSON responses
- endpoints
- mobile apps and dashboards call it

GET Reads, POST Creates

We only need two HTTP methods for this beginner project.

GET

- read data
- GET /health
- GET /tickets
- GET /tickets/1
- should not create records

POST

- create data
- POST /tickets
- sends JSON body
- returns new ticket
- status code 201

APIs Often Send JSON

JSON is a structured text format used for data exchange.

```
{  
  "student_name": "Ari",  
  "category": "network",  
  "priority": "high",  
  "issue": "wifi drops during class"  
}
```

Keys and string values use double quotes.

Virtual Environment

A virtual environment keeps project packages separate.

```
$ python3 -m venv .venv  
$ source .venv/bin/activate  
$ python -m pip install --upgrade pip  
$ python -m pip install fastapi uvicorn
```

If installs fail, use the written analysis option.

FastAPI Creates Endpoints

Each decorator maps a URL path to a Python function.

```
from fastapi import FastAPI
```

```
app = FastAPI(title="Campus Helpdesk Ticket Tracker")
```

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

Decorator plus function equals endpoint.

Uvicorn Starts The Server

The server listens for local web requests.

```
$ uvicorn main:app --reload
```

Open:
<http://127.0.0.1:8000/docs>

Stop server:
Ctrl + C

--reload restarts when code changes.

The /docs Page Is A Testing Tool

FastAPI gives us an interactive API page without building a frontend.

Use /docs to

- see endpoints
- run GET /health
- run GET /tickets
- send POST JSON
- copy responses

Submit evidence

- screenshot of docs
- health response
- POST request and response
- ticket list
- error if one occurred

Create Data Through The API

POST sends a request body to the backend.

```
@app.post("/tickets", status_code=201)
def create_ticket(ticket: TicketCreate):
    new_ticket = ticket.dict()
    new_ticket["id"] = len(tickets) + 1
    new_ticket["status"] = "open"
    tickets.append(new_ticket)
    return new_ticket
```

The starter version uses an in-memory list.

Move From Memory To Persistence

The final version stores tickets in tickets.db.

with get_connection() as connection:

```
    cursor = connection.execute(
        "INSERT INTO tickets (...) VALUES (?, ?, ?, ?, ?)",
        (student_name, category, priority, "open", issue),
    )
    connection.commit()
```

This connects Class 03 to the API.

Test From The Terminal Too

curl sends web requests from the command line.

```
$ curl http://127.0.0.1:8000/health
$ curl http://127.0.0.1:8000/tickets
$ curl -X POST http://127.0.0.1:8000/tickets \
  -H "Content-Type: application/json" \
  -d '{"student_name":"Blake","category":"software","priority":"normal","issue":"Python question"}
```

Docs page is easier. curl is useful job practice.

Read The Error Before Changing Code

Debugging is evidence collection, not guessing.

Ask

- what command ran?
- did the server start?
- what file is named?
- what line number?
- what request failed?
- does tickets.db exist?

Common causes

- wrong folder
- venv not active
- package missing
- syntax typo
- port already in use
- bad JSON

Prove The Database Works

Create a ticket, restart the server, and check that it remains.

1. Start the app
2. POST a new ticket
3. GET /tickets and confirm it appears
4. Stop server with Ctrl+C
5. Start server again
6. GET /tickets again
7. Ticket still appears = persistence

This is strong Brightspace evidence.

Explain It Like An Entry-Level Candidate

The explanation should be honest, concrete, and plain.

I built a small helpdesk ticket API on Linux.

I used Python and FastAPI for the backend endpoints.

I used SQLite so tickets persisted after restart.

I tested the app with the /docs page and terminal commands.

I debugged errors by reading server output and checking files.

This is not exaggeration. It is a clear beginner project story.

Brightspace Submission

Students submit code, API evidence, persistence evidence, and reflection.

Evidence

- /docs screenshot
- GET /health output
- POST request and response
- GET /tickets output
- persistence proof
- final main.py

Writing

- what each endpoint does
- what SQLite does
- one debugging note
- job role connection
- interview explanation

If Installs Fail, Do The Analysis Option

Do not let package installation consume the entire class.

Acceptable evidence

- instructor demo notes
- code reading answers
- screenshots from demo
- written endpoint explanations
- honest error screenshot

Still learning

- API purpose
- GET vs POST
- JSON body
- SQLite persistence
- debugging workflow

What Success Looks Like

Students can run or explain a small Linux-backed API project.

Students should be able to say

- I ran a FastAPI app
- I tested /docs
- I created a ticket
- I understand GET and POST
- I know why SQLite matters

The story

- Linux environment
- Python backend
- API endpoints
- database persistence
- debugging evidence